CUDA Streams & Asynchronous Execution

학습 목표

- CUDA Streams 개념 이해
- Asynchronous execution 활용
- Concurrent kernel execution
- Host-Device overlap
- 성능 최적화 전략

예상 학습 시간: 3-4시간
난이도: ⭐ ⭐ ⭐ (중급-고급)
선수 과목: 00-04 CUDA 기초 및 중급

---

1. 왜 Streams인가?

1.1 Synchronous vs Asynchronous

문제: Default Synchronous Execution

```
// 전통적인 CUDA 코드
cudaMemcpy(d_data, h_data, size, cudaMemcpyHostToDevice);  // 1. Copy
kernel<<<grid, block>>>(d_data);                            // 2. Compute
cudaMemcpy(h_result, d_result, size, cudaMemcpyDeviceToHost); // 3. Copy back

// Timeline:
// CPU: ──────┬──────────┬──────────┬──────
//            H2D       Kernel    D2H
// GPU:       [▓▓▓▓]    [▓▓▓▓]    [▓▓▓▓]
//            ↑ CPU waits here (idle!)
```

문제점:

- CPU가 GPU 완료를 기다림 (blocking)
- GPU 리소스가 idle 상태로 낭비
- 여러 작업을 동시에 처리 불가

1.2 해결: Asynchronous Execution

```
// Stream을 사용한 비동기 실행
cudaStream_t stream1, stream2;
cudaStreamCreate(&stream1);
cudaStreamCreate(&stream2);

// Stream 1: Data A 처리
cudaMemcpyAsync(d_A, h_A, size, H2D, stream1);
kernel<<<grid, block, 0, stream1>>>(d_A);
cudaMemcpyAsync(h_A_result, d_A, size, D2H, stream1);

// Stream 2: Data B 처리 (동시에!)
cudaMemcpyAsync(d_B, h_B, size, H2D, stream2);
kernel<<<grid, block, 0, stream2>>>(d_B);
cudaMemcpyAsync(h_B_result, d_B, size, D2H, stream2);

// Timeline:
// Stream1: [H2D-A][Kernel-A][D2H-A]
// Stream2:     [H2D-B][Kernel-B][D2H-B]
//          ▲ Overlapping execution! 🚀
```

---

2. CUDA Streams 기초

2.1 Stream이란?

Stream: GPU 연산들의 순서가 있는 큐 (FIFO)

```
Stream 1: [Op1] → [Op2] → [Op3] → ...
          순서 보장

Stream 2: [Op4] → [Op5] → [Op6] → ...
          순서 보장

Stream 1과 Stream 2는 독립적으로 실행 (순서 보장 안 됨)
```

2.2 Stream 생성 및 관리

```
// Stream 생성
cudaStream_t stream;
cudaStreamCreate(&stream);

// Async 연산에 stream 전달
cudaMemcpyAsync(dst, src, size, kind, stream);
kernel<<<grid, block, smem, stream>>>(args);

// Stream 동기화 (해당 stream만 대기)
cudaStreamSynchronize(stream);

// 모든 streams 동기화
cudaDeviceSynchronize();

// Stream 파괴
cudaStreamDestroy(stream);
```

2.3 Default Stream (Stream 0)

```
// Stream 지정 안 하면 default stream 사용
kernel<<<grid, block>>>(args);
// 동일:
kernel<<<grid, block, 0, 0>>>(args);

// Default stream 특징:
// - Synchronous (blocking)
// - 모든 device 연산과 serialize됨
```

---

3. Concurrent Kernel Execution

3.1 요구사항

하드웨어 요구사항:

- Compute Capability 2.0 이상
- Multiple kernel 동시 실행 지원

확인 방법:

```
cudaDeviceProp prop;
cudaGetDeviceProperties(&prop, 0);
printf("Concurrent kernels: %d\n", prop.concurrentKernels);
printf("Async engine count: %d\n", prop.asyncEngineCount);
// asyncEngineCount: 2 = 양방향 동시 복사 지원
```

3.2 Concurrent Kernel 예제

```
#define N_STREAMS 4

cudaStream_t streams[N_STREAMS];
for (int i = 0; i < N_STREAMS; i++) {
    cudaStreamCreate(&streams[i]);
}

// 4개 kernels 동시 실행
for (int i = 0; i < N_STREAMS; i++) {
    kernel<<<grid, block, 0, streams[i]>>>(d_data + i * chunk_size);
}

// 모든 streams 완료 대기
for (int i = 0; i < N_STREAMS; i++) {
    cudaStreamSynchronize(streams[i]);
}
```

성능 향상:

```
Sequential:  4 × 10ms = 40ms
Concurrent:  max(10ms) = 10ms (각 kernel이 SM의 25% 사용 시)
Speedup: 4x! 🚀
```

3.3 SM 활용도

```
// 예: RTX 4090 (128 SMs)

// Kernel 1: 32 blocks × 256 threads
// → 32 SMs 사용 (25%)
```

```
// 동시에 4개 kernels 실행 가능!
// 32 + 32 + 32 + 32 = 128 SMs (100% 활용)
```

---

4. Memory Copy & Compute Overlap

4.1 Pinned Memory (Page-Locked)

일반 메모리 (Pageable):

```
// malloc으로 할당 (pageable)
float* h_data = (float*)malloc(size);

// 문제: OS가 메모리 페이지를 swap out 가능
// → GPU가 접근하기 전에 pinning 필요 (blocking!)
```

Pinned Memory:

```
// CUDA가 관리 (page-locked)
float* h_data;
cudaMallocHost(&h_data, size);  // 또는 cudaHostAlloc()

// 장점:
// 1. DMA (Direct Memory Access) 사용 가능
// 2. Async copy 가능 (cudaMemcpyAsync)
// 3. Faster copy (2-3x)

// 단점:
// - 시스템 메모리 제한 (너무 많이 사용 금지)
// - 일반 CPU 접근은 약간 느림
```

4.2 Overlap Pattern

```
cudaStream_t stream;
cudaStreamCreate(&stream);

// Pinned memory 할당
float *h_data, *h_result;
cudaMallocHost(&h_data, size);
cudaMallocHost(&h_result, size);
```

```
// Async operations
cudaMemcpyAsync(d_data, h_data, size, H2D, stream);
kernel<<<grid, block, 0, stream>>>(d_data);
cudaMemcpyAsync(h_result, d_result, size, D2H, stream);

// CPU는 블록 안 됨! 다른 작업 가능
do_cpu_work();

// 나중에 결과 필요할 때 동기화
cudaStreamSynchronize(stream);
```

4.3 Multi-Stream Overlap

```
// Data를 N개 chunks로 분할
int N_CHUNKS = 4;
int chunk_size = total_size / N_CHUNKS;

for (int i = 0; i < N_CHUNKS; i++) {
    int offset = i * chunk_size;

    // Stream i: Copy → Compute → Copy back
    cudaMemcpyAsync(d_data + offset, h_data + offset,
                    chunk_size, H2D, streams[i]);

    kernel<<<grid, block, 0, streams[i]>>>(d_data + offset, chunk_size);

    cudaMemcpyAsync(h_result + offset, d_result + offset,
                    chunk_size, D2H, streams[i]);
}

// Timeline:
// Stream 0: [H2D-0][Kernel-0][D2H-0]
// Stream 1:    [H2D-1][Kernel-1][D2H-1]
// Stream 2:       [H2D-2][Kernel-2][D2H-2]
// Stream 3:          [H2D-3][Kernel-3][D2H-3]
//           ▲ Copy와 Compute overlap!
```

---

5. Stream Priorities

5.1 우선순위 설정

```
// 우선순위 범위 확인
int priority_low, priority_high;
cudaDeviceGetStreamPriorityRange(&priority_low, &priority_high);
// 예: low = 0, high = -1 (숫자 작을수록 높은 우선순위)

// 우선순위가 있는 stream 생성
cudaStream_t high_priority_stream, low_priority_stream;
cudaStreamCreateWithPriority(&high_priority_stream, cudaStreamNonBlocking, prior
cudaStreamCreateWithPriority(&low_priority_stream, cudaStreamNonBlocking, priori

// 중요한 작업은 high priority stream에
critical_kernel<<<grid, block, 0, high_priority_stream>>>(args);

// 덜 중요한 작업은 low priority stream에
background_kernel<<<grid, block, 0, low_priority_stream>>>(args);
```

사용 사례:

- 실시간 rendering (high) vs 백그라운드 계산 (low)
- Latency-critical vs Throughput-oriented

5.2 Non-Blocking Streams

```
// Default: cudaStreamDefault (blocking)
cudaStream_t stream1;
cudaStreamCreate(&stream1);

// Non-blocking stream
cudaStream_t stream2;
cudaStreamCreateWithFlags(&stream2, cudaStreamNonBlocking);

// Difference:
// - Default stream과의 implicit synchronization 여부
// - Non-blocking: 다른 streams와 완전 독립
```

---

6. Stream Synchronization

6.1 동기화 방법들

```
// 1. Stream 단위 동기화 (해당 stream만 대기)
cudaStreamSynchronize(stream);

// 2. Device 전체 동기화 (모든 streams 대기)
cudaDeviceSynchronize();

// 3. Event 기반 동기화 (더 세밀한 제어)
cudaEvent_t event;
cudaEventCreate(&event);
cudaEventRecord(event, stream1);
cudaStreamWaitEvent(stream2, event, 0);  // stream2가 event 대기

// 4. Query (non-blocking check)
cudaError_t status = cudaStreamQuery(stream);
if (status == cudaSuccess) {
    // Stream의 모든 작업 완료
} else if (status == cudaErrorNotReady) {
    // 아직 실행 중
}
```

6.2 Event를 사용한 Dependencies

```
cudaEvent_t event1, event2;
cudaEventCreate(&event1);
cudaEventCreate(&event2);

// Stream 1: Phase A
kernel_phase_A<<<grid, block, 0, stream1>>>(d_data);
cudaEventRecord(event1, stream1);  // Marker

// Stream 2: Phase B (Phase A 완료 대기)
cudaStreamWaitEvent(stream2, event1, 0);
kernel_phase_B<<<grid, block, 0, stream2>>>(d_data);
cudaEventRecord(event2, stream2);

// Stream 3: Phase C (Phase B 완료 대기)
cudaStreamWaitEvent(stream3, event2, 0);
kernel_phase_C<<<grid, block, 0, stream3>>>(d_data);
```

DAG (Directed Acyclic Graph) 실행:

```
    Phase A (stream1)
       ↓
    Phase B (stream2)
       ↓
    Phase C (stream3)
```

---

7. Performance Analysis

7.1 Timeline 분석 (Nsight Systems)

```
# Profile with Nsight Systems
nsys profile -o report ./program

# View timeline
nsys-ui report.qdrep
```

보는 것:

- Kernel execution overlap
- Memory copy overlap
- CPU-GPU bubbles (idle time)
- Stream utilization

7.2 성능 측정

```
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);

// 전체 시간 측정
cudaEventRecord(start);

// Multiple streams 작업
for (int i = 0; i < N_STREAMS; i++) {
    cudaMemcpyAsync(..., streams[i]);
    kernel<<<..., streams[i]>>>(...);
    cudaMemcpyAsync(..., streams[i]);
}
```

```
cudaEventRecord(stop);
cudaEventSynchronize(stop);

float milliseconds;
cudaEventElapsedTime(&milliseconds, start, stop);
printf("Total time: %.3f ms\n", milliseconds);
```

7.3 최적 Stream 개수

```
// 실험적으로 찾기
for (int n_streams = 1; n_streams <= 16; n_streams *= 2) {
    float time = benchmark_with_n_streams(n_streams);
    printf("Streams: %d, Time: %.3f ms\n", n_streams, time);
}

// 일반적인 최적값:
// - Compute-bound: 2-4 streams
// - Memory-bound: 4-8 streams
// - Too many streams: Overhead 증가
```

---

8. Best Practices

8.1 DO's

```
// ✅ 1. Pinned memory 사용 (async copy)
cudaMallocHost(&h_data, size);

// ✅ 2. 적절한 chunk size
// - 너무 작으면: Overhead
// - 너무 크면: Overlap 감소
int chunk_size = total_size / 8;  // 경험적

// ✅ 3. Stream 재사용
cudaStream_t streams[N_STREAMS];
// 한 번 생성, 여러 번 사용

// ✅ 4. Non-blocking streams
cudaStreamCreateWithFlags(&stream, cudaStreamNonBlocking);
```

8.2 DON'Ts

```
// ❌ 1. 너무 많은 streams
// Overhead로 인해 느려질 수 있음
for (int i = 0; i < 1000; i++) {  // BAD
    cudaStreamCreate(&streams[i]);
}

// ❌ 2. Default stream 사용 (blocking)
kernel<<<grid, block>>>(args);  // BAD for concurrency

// ❌ 3. Synchronization 남용
for (int i = 0; i < N; i++) {
    kernel<<<..., stream>>>(args);
    cudaStreamSynchronize(stream);  // BAD: Kills concurrency
}

// ❌ 4. Pageable memory with Async
float* h_data = malloc(size);
cudaMemcpyAsync(d_data, h_data, size, H2D, stream);  // BAD: Falls back to sync
```

---

9. 고급 패턴

9.1 Producer-Consumer

```
// Producer stream: Data 준비
cudaMemcpyAsync(d_input, h_input, size, H2D, stream_producer);
preprocess<<<..., stream_producer>>>(d_input);
cudaEventRecord(event_ready, stream_producer);

// Consumer stream: Data 처리 (ready 대기)
cudaStreamWaitEvent(stream_consumer, event_ready, 0);
process<<<..., stream_consumer>>>(d_input);
```

9.2 Double Buffering

```
float *h_buffer[2], *d_buffer[2];
cudaStream_t streams[2];
```

```
// Overlap: Process buffer 0 while loading buffer 1
for (int i = 0; i < num_iterations; i++) {
    int current = i % 2;
    int next = (i + 1) % 2;

    // Load next data (async)
    cudaMemcpyAsync(d_buffer[next], h_buffer[next], size, H2D, streams[next]);

    // Process current data
    kernel<<<..., streams[current]>>>(d_buffer[current]);

    // Copy result back
    cudaMemcpyAsync(h_buffer[current], d_buffer[current], size, D2H, streams[cur
}
```

9.3 Pipeline Pattern

```
// 3-stage pipeline: Load → Compute → Store
for (int i = 0; i < N; i++) {
    // Stage 1: Load
    cudaMemcpyAsync(d_in[i%3], h_in[i], size, H2D, streams[0]);

    // Stage 2: Compute (previous data)
    if (i > 0) {
        kernel<<<..., streams[1]>>>(d_in[(i-1)%3], d_out[(i-1)%3]);
    }

    // Stage 3: Store (data from 2 iterations ago)
    if (i > 1) {
        cudaMemcpyAsync(h_out[i-2], d_out[(i-2)%3], size, D2H, streams[2]);
    }
}
```

---

10. 실습 예제

다음 파일 참고:

- `examples/cuda/07_streams_basic.cu` - Stream 기초
- `examples/cuda/08_streams_overlap.cu` - Copy-Compute overlap

- `examples/cuda/09_streams_concurrent.cu` - Concurrent kernels

요약 체크리스트

- [ ] Stream 개념 이해
- [ ] Async vs Sync 차이 파악
- [ ] Pinned memory 사용 이유 이해
- [ ] Concurrent kernel execution 구현 가능
- [ ] Copy-Compute overlap 패턴 숙지
- [ ] Event 기반 동기화 사용 가능
- [ ] Stream priority 활용법 이해
- [ ] Nsight Systems로 timeline 분석 가능

핵심 개념:

- Streams = Independent execution queues
- Pinned memory = Required for async
- Overlap = Copy while computing
- Events = Fine-grained synchronization
- Optimal streams = 2-8 (실험 필요)